MODULE

Introduction à l'informatique

Objectifs, contenu et organisation du module

1re année géomatique

proposition pour 2027-2028

Objectif du cours

Consolider ou acquérir les bases informatiques nécessaires aux autres enseignements, en particulier ceux de programmation et les TD utilisant Python.

Demandé dans les autres modules et cours	(re)vu dans ce module
« ouvrez le projet fourni »	travailler dans un éditeur de code, lire une arborescence
« installez Python et numpy »	créer un environnement et le réinstaller ailleurs
« le script lit `donnees.csv` »	manipuler des fichiers depuis Python
« rendez votre code »	versionner avec git, partager un dépôt, documentation README.md

Objectifs du module liés à la programmation

Maîtriser la forme d'un projet de code : documentation, organisation des fichiers, bibliothèques et environnements.

	ce cours	cours de programmation
Quel algorithme choisir ?		oui
Comment écrire cette boucle ?		oui
Où mettre ce fichier ?	oui	
Comment lancer le script ailleurs ?	oui	

Les trois compétences du module

Éditer	Versionner	Structurer
éditeur de code, arborescence de projet	git : enregistrer, revenir, partager	README, environnement, ligne de commande

Des notions de culture informatique au fil de l'eau, rattachées aux enseignements et projets en cours.

Culture informatique	ordres de grandeur, sécurité informatique, du programme au logiciel

Organisation : sept séances de deux heures

Chaque séance alterne explications courtes et TD sur machine.

	Sujet	Type
1	Logiciels, fichiers, terminal et premier programme	cours
2	Éditeur de code, Markdown et git local	cours
3	Environnements, fichiers en Python, ligne de commande	cours
4	Premier projet : une animation, du notebook au programme	projet
5	Matériel, réseau ; mots de passe, clés, secrets	cours
6	La forge, sur le dépôt du projet 4	cours
7	Second projet : un effet pour l'animation	projet

Attention. Une partie de la séance 1 reprend des notions surement vues au lycée et/ou prépa, et avance vite : posez vos questions tout de suite.

Cours 1

Version 2, proposition de travail

1re année géomatique
2027-2028

Logiciels, fichiers et stockage

Ce qu'un logiciel fait, le rôle des fichiers et les disques où ils sont enregistrés.

Logiciel : définition et termes courants

Définition : logiciel

Ensemble des programmes, procédés et règles, et éventuellement de la documentation, relatifs au fonctionnement d'un ensemble de traitement de données.

Source : *Journal officiel* du 22/09/2000 (vocabulaire de l'informatique). Le seul de ces mots à avoir une définition officielle.

Autres termes courants, qui désignent chacun un type de logiciel.

application	**app**	**webapp**	**OS**	**driver**

Les définitions d'usage : Grand dictionnaire terminologique de l'OQLF.

Le système d'exploitation

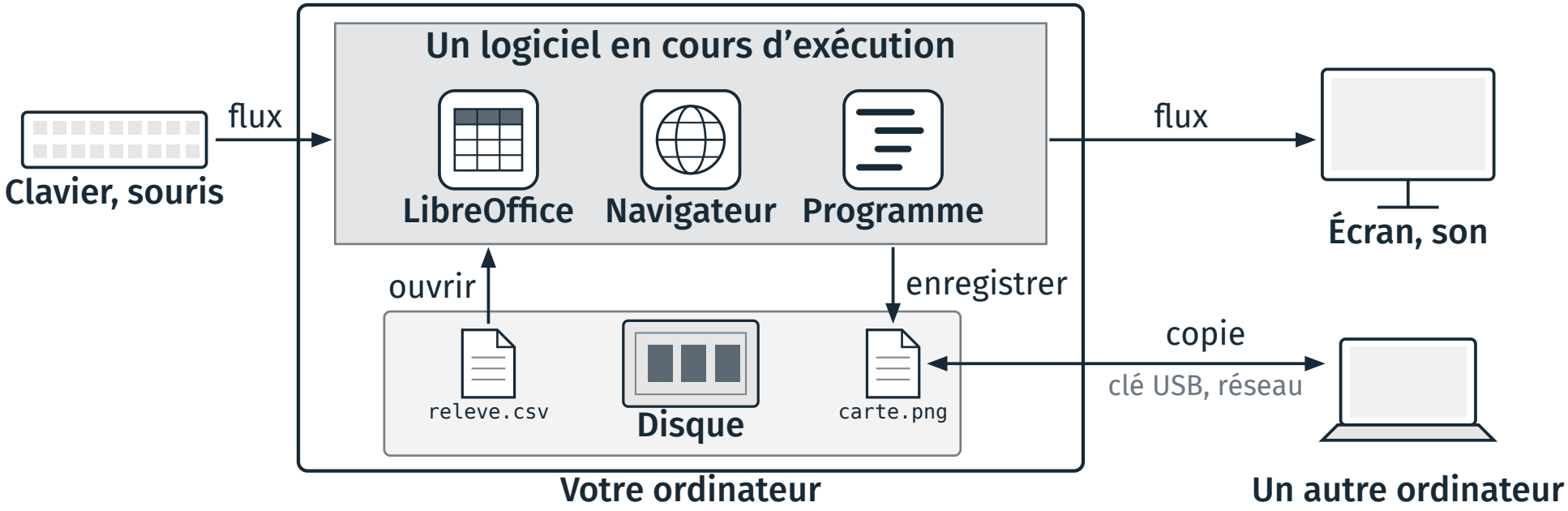
Un programme ne s’adresse pas directement au matériel : il passe par le système.



Quel système d’exploitation tourne sur votre téléphone ?

Entrées et sorties d'un logiciel

Les entrées et sorties d’un logiciel sont des fichiers ou des flux échangés avec les périphériques. Un fichier conserve un résultat, que l’on peut ensuite ouvrir avec un autre logiciel, copier sur une autre machine ou remettre à quelqu’un d’autre.



Fichier, extension et type de fichier

Le type d'un fichier (texte, vidéo…) est indiqué par son **extension**. Le système s'en sert pour choisir le logiciel à lancer.

releve_2026**.csv**

le nom, que vous choisissez l'extension

> **Attention.** Windows masque les extensions qu'il connaît : `raven.odt` s'affiche `raven`. Réglage à changer une fois, avant le TD 1a.

Reconnaître un format à son extension

Pour chacune de ces extensions, dites de quel type de contenu il s'agit, et si le fichier est lisible dans un éditeur de texte.

`.mp3`	`.mp4`	`.jpg`	`.png`
`.svg`	`.tif`	`.pdf`	`.odt`
`.xlsx`	`.csv`	`.zip`	`.exe`
`.py`	`.md`	`.json`	`.yaml`

Reconnaître un format à son extension — réponses

`.mp3` son, avec perte	`.mp4` vidéo, la plus courante	`.jpg` photo, avec perte	`.png` image, sans perte
`.svg` image vectorielle : texte	`.tif` image, y compris GeoTIFF	`.pdf` document mis en page	`.odt` LibreOffice, archive ZIP
`.xlsx` Excel, archive ZIP	`.csv` tableau : du texte	`.zip` archive de fichiers	`.exe` programme Windows
`.py` code Python : du texte	`.md` documentation : du texte	`.json` données, réglages : texte	`.yaml` réglages : du texte

Six de ces seize formats sont du texte : ceux qu'on peut ouvrir dans un éditeur, comparer ligne à ligne et versionner. En bleu, les quatre que vous écrirez vous-mêmes dans ce module.

Quizz : vocabulaire associé aux chemins de fichier

Un chemin dit où trouver un fichier dans l'arborescence des dossiers. Plusieurs mots en désignent les parties : dites à quoi chacun correspond dans cet exemple.

```
C:\Users\alice\Documents\raven.odt
```

Le mot	Ce qu'il désigne dans l'exemple
la racine, ou le disque	
un nom de dossier	
le nom du fichier	
le chemin du fichier	
le dossier parent	

Quizz : vocabulaire associé aux chemins de fichier - Réponse

`C:\Users\alice\Documents\raven.odt`

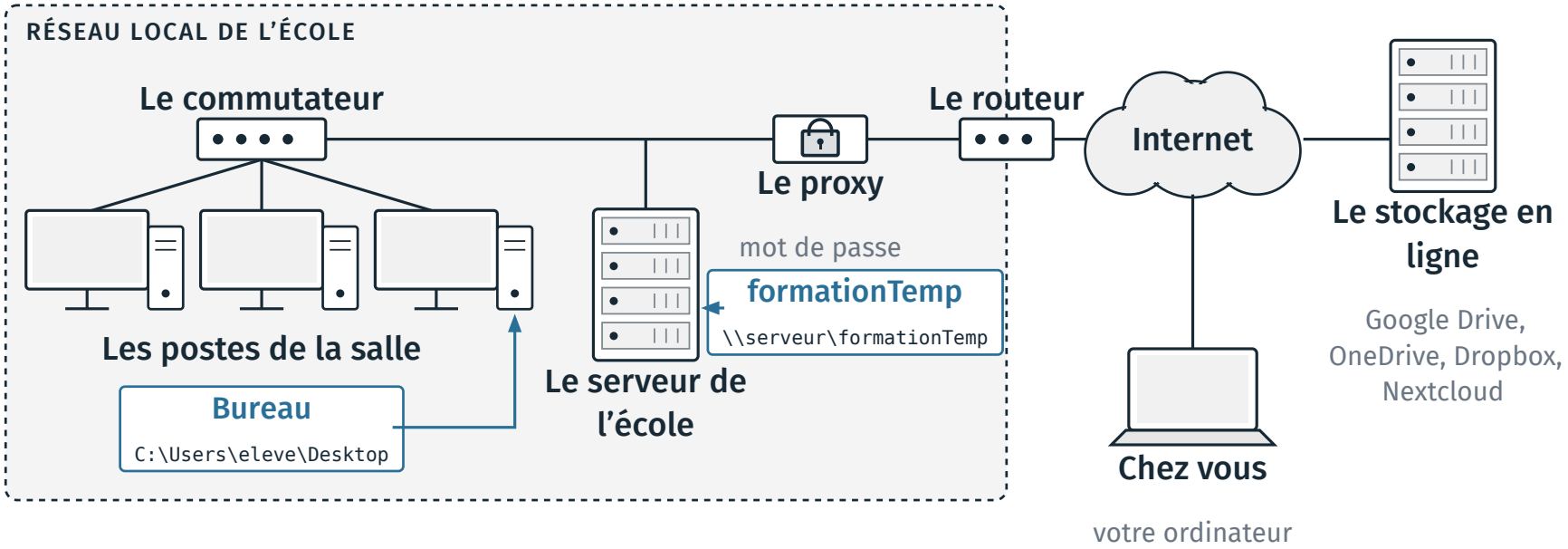
la racine, ou le disque — trois noms de dossier — le nom du fichier, extension comprise

Le mot	Ce qu'il désigne dans l'exemple
la racine, ou le disque	`C:\`, le point de départ ; `/` sous macOS et Linux
un nom de dossier	`Users`, `alice`, `Documents` : trois, du plus large au plus précis
le nom du fichier	`raven.odt`, extension comprise
le chemin du fichier	tout, de la racine au fichier
le dossier parent	`C:\Users\alice\Documents`, le dossier qui le contient

Un chemin se lit de gauche à droite, de la racine au fichier ; chaque séparateur descend d'un dossier.

Les emplacements de stockage sur le réseau

Le Bureau est sur le disque du poste, `formationTemp` sur le disque du serveur de l'école. Pour aller sur Internet, les postes passent par le proxy de l'école, qui demande un mot de passe.



Un chemin qui commence par `\\` désigne un dossier d'une autre machine, ici le serveur.

TD 1a

Fichiers, formats et extensions

Objectif : récupérer les fichiers de la séance, distinguer l'extension d'un fichier et son contenu sur un même texte enregistré dans plusieurs formats, puis modifier à plusieurs un fichier du dossier partagé

`cours1/1a_formats/` ≈ 15′

Les fichiers du cours

L'archive de la séance, `info01-cours1.zip`, est dans `formationTemp`. Chacun doit la copier sur le Bureau du poste, puis l'extraire.

Copie	Emplacement	Qui y a accès
de	`formationTemp` : serveur de l'école	toute la promotion
vers	Bureau : disque du poste	votre session, sur ce poste

```
cours1/                un dossier par TD, dans l'ordre de la séance
├── 1a_formats/
├── 1b_terminal/
├── 1c_programme/
└── 1d_notebook/
```

Dans chaque dossier, la feuille du TD et son guide détaillé, en PDF.

Copier les fichiers de la séance sur le poste

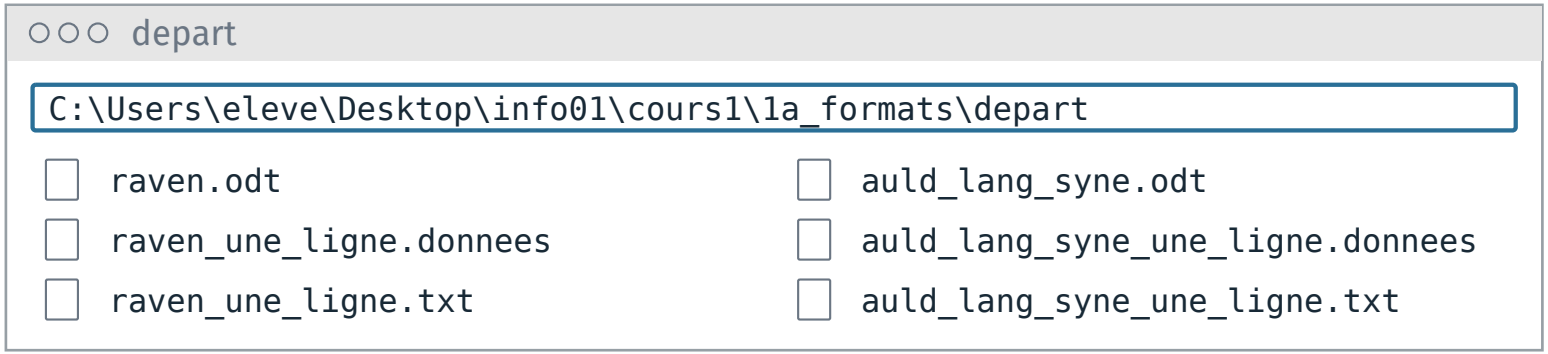
1. Ouvrir `formationTemp` : double-clic sur son raccourci, sur le Bureau.
2. Copier `info01-cours1.zip` : clic droit sur l'archive, Copier.
3. Créer le dossier `info01` sur le Bureau, s'il n'existe pas : clic droit sur le Bureau, Nouveau, Dossier.
4. Coller l'archive dans `info01` : ouvrir `info01`, puis clic droit, Coller.
5. Extraire l'archive : clic droit sur `info01-cours1.zip`, Extraire tout…
6. Extraire dans `info01` même : dans la fenêtre qui s'ouvre, effacer `\info01-cours1` à la fin du chemin, puis cliquer sur Extraire.
7. Ouvrir `cours1\1a_formats\`, et vérifier qu'il contient `depart\` et `travail\`.

Le détail, avec les messages de Windows : page « Récupérer les fichiers d'une séance » du site du cours.

Deux dossiers : `depart/` et `travail/`

> **Attention.** Avant d'ouvrir un fichier : afficher les extensions, que Windows masque par défaut. Explorateur → Affichage → Afficher → Extensions de noms de fichiers.

Ne pas modifier les fichiers de `depart/`. Enregistrer dans `travail/` les fichiers que vous produisez.

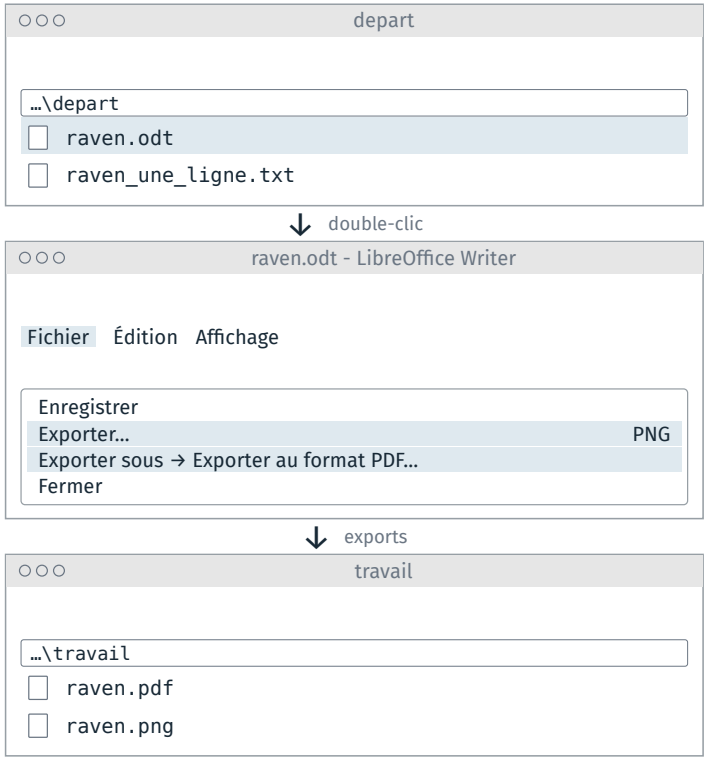


← **Vérifier ici le dossier ouvert**

Exporter le document de départ

1. Ouvrir `raven.odt` avec LibreOffice Writer : dans `depart\`, double-clic sur `raven.odt`.
2. Exporter le document en PDF, dans `travail\`.
   a. Menu Fichier, Exporter sous, Exporter au format PDF…, puis Exporter.
   b. La fenêtre d'enregistrement affiche `depart\`. Cliquer sur la flèche vers le haut, double-cliquer sur `travail`, puis Enregistrer.
3. Exporter le document en PNG, dans `travail\`.
   a. Menu Fichier, Exporter…, puis choisir le type PNG.
   b. Vérifier que la fenêtre affiche `travail\`, puis Enregistrer.
4. Fermer Writer sans enregistrer `raven.odt`.

Le détail, avec les fenêtres de LibreOffice : guide du TD 1a, étape 3.



Un même document, trois formats

Rouvrir les trois fichiers, et essayer dans chacun de sélectionner une ligne du poème, puis de chercher un mot avec `Ctrl` + `F`.

Fichier	Origine
`depart/raven.odt`	le document de départ
`travail/raven.pdf`	l'export en PDF
`travail/raven.png`	l'export en image

Rouvrir les trois dans LibreOffice : le `.png` s'ouvre dans Draw, comme une image posée sur une page.

Ouvrir les fichiers avec un éditeur de texte

On ouvre cette fois les fichiers avec un éditeur de texte, le Bloc-notes ou Notepad++. LibreOffice ne sert pas dans cette étape.

1. Ouvrir `raven_une_ligne.txt` avec le Bloc-notes ou Notepad++ : dans `depart\`, clic droit sur le fichier, Ouvrir avec, puis l'éditeur choisi.
2. Ouvrir de la même façon `raven_une_ligne.donnees`, puis `raven.odt`, et comparer avec `raven_une_ligne.txt`.
3. Fermer chaque fenêtre sans enregistrer.



Bloc-notes

ou



Notepad++

Attention. Un `.odt` enregistré depuis le Bloc-notes ou Notepad++ est abîmé, et Writer ne peut plus l'ouvrir.

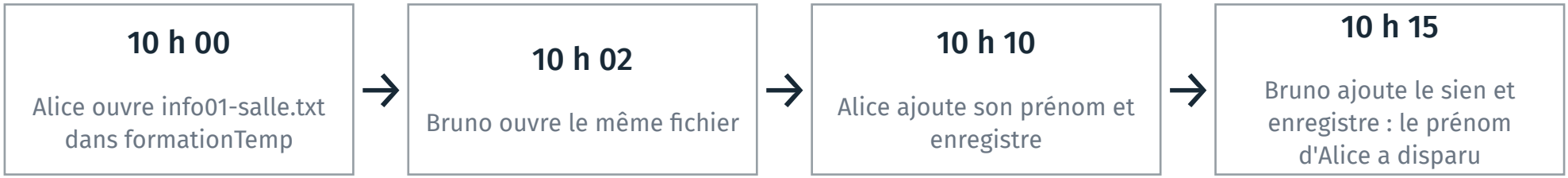
Modifier à plusieurs un fichier partagé

Toute la salle ajoute une ligne au même fichier, `formationTemp\info01-salle.txt`. Ne modifier aucun autre fichier de `formationTemp`.

	Ce qu'il faut faire
1	dans `formationTemp`, clic droit sur `info01-salle.txt`, Ouvrir avec, Bloc-notes ; attendre le signal
2	au signal, écrire son prénom, puis `Ctrl` + `S`
3	fermer le Bloc-notes, puis rouvrir le fichier

Modifications simultanées d'un fichier partagé

Un fichier du dossier partagé est le même pour toute la salle. Avec un éditeur de texte, le dernier qui enregistre remplace la version de l'autre.



Attention. Ne pas travailler dans `formationTemp`. Copier les fichiers sur le poste, et travailler dans la copie.

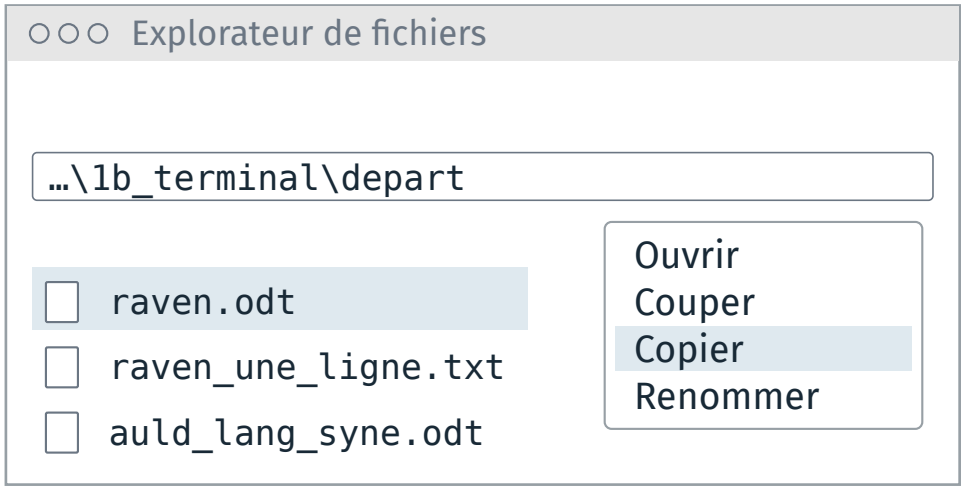
Le terminal

Se déplacer dans les dossiers et agir sur les fichiers en tapant des commandes.

Interface graphique et ligne de commande

Dans une interface graphique, on choisit l'opération parmi celles que la fenêtre affiche. En ligne de commande, on tape le nom de l'opération, qu'il faut connaître.

Interface graphique : l'explorateur de fichiers



La copie demande ensuite d'ouvrir `travail`, puis de choisir Coller au clic droit.

Ligne de commande : Git Bash



On tape la ligne, puis on appuie sur Entrée. `cp` abrège *copy*.

Le terminal et l'interpréteur de commandes

Le terminal est la fenêtre : il affiche du texte et transmet ce qu'on tape. L'interpréteur de commandes lit chaque ligne et lance le programme qu'elle nomme.



Git Bash

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1b_terminal
$ ls depart
auld_lang_syne.odt                raven.odt
auld_lang_syne_une_ligne.donnees  raven_une_ligne.donnees
auld_lang_syne_une_ligne.txt      raven_une_ligne.txt

eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1b_terminal
$
```

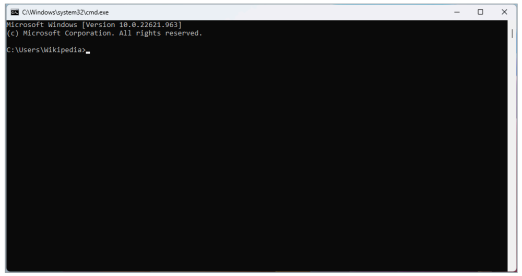
Le mot « terminal » désigne souvent les deux à la fois. Le langage des commandes est celui de l'interpréteur : cmd, PowerShell ou bash.

Les trois terminaux du poste

L'invite, au début de chaque ligne, indique quel interpréteur lit la commande.

Invite de commandes (cmd)

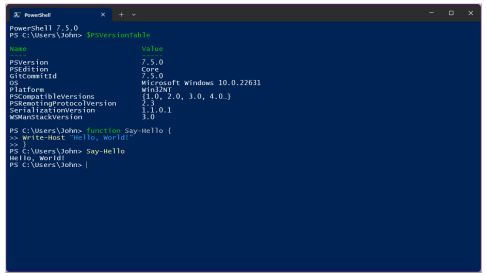
Démarrer → `cmd`



Invite : `C:\Users\…>`

PowerShell

Démarrer → Terminal

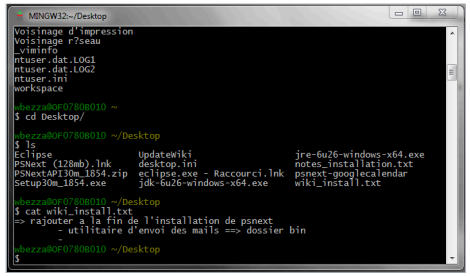


Invite : `PS C:\Users\…>`

Windows

bash / Git Bash

clic droit → Open Git Bash here



Invite : `utilisateur@machine`, puis `$`

Linux, macOS ; Windows avec Git Bash

Le module emploie Git Bash. Git Bash est installé avec git sur les postes. Son interpréteur, bash, est celui de Linux et de la plupart des serveurs, et les mêmes commandes fonctionnent sous macOS.

L'invite de Git Bash

Avant chaque commande, Git Bash affiche une invite : l'utilisateur, la machine et le dossier courant. La commande se tape après le `$`.

MINGW64:/c/Users/eleve/Desktop/info01/cours1

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1
$ ls
1a_formats/ 1b_terminal/ 1c_programme/ 1d_notebook/

eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1
$
```

Dans l'invite	Ce qu'il désigne
`eleve`	l'utilisateur
`POSTE-12`	la machine
`MINGW64`	Git Bash sous Windows
`~`	le dossier personnel, `C:\Users\eleve`
`~/Desktop/…`	le dossier courant
`$`	l'attente d'une commande

Le `/` à la fin d'un nom signale un dossier. Après le TD 1c, une ligne `(base)` s'ajoute au-dessus : l'environnement conda actif.

La forme d'une commande

Une commande commence par le nom du programme à lancer. Suivent des options, qui commencent par un tiret, et des arguments, souvent des noms de fichiers.

Commande tapée	Programme	Options	Arguments
`ls`	`ls`		
`ls -a depart`	`ls`	`-a`	`depart`
`cp depart/raven.odt travail/`	`cp`		`depart/raven.odt`, `travail/`
`python --version`	`python`	`--version`	
`ls --help`	`ls`	`--help`	

Une option d'une lettre prend un tiret, une option en toutes lettres en prend deux. `--help` affiche l'aide de la plupart des commandes.

Les commandes de base : se déplacer

Chaque commande de base a son équivalent à la souris dans l'explorateur de fichiers.

Commande	Ce qu'elle fait	Dans l'explorateur
`pwd`	affiche le dossier courant	la barre d'adresse
`ls`	liste le dossier courant	la fenêtre ouverte
`ls dossier`	liste le dossier nommé	ouvrir ce dossier
`cd dossier`	descend dans le dossier nommé	double-clic sur le dossier
`cd ..`	remonte au dossier parent	flèche « dossier parent »

Les commandes de base : agir sur les fichiers

Commande	Ce qu'elle fait	Dans l'explorateur
`cp source destination`	copie un fichier	`Ctrl` + `C`, puis `Ctrl` + `V`
`mv source destination`	déplace ou renomme	glisser ; `F2`
`mkdir nom`	crée un dossier	Nouveau dossier
`rm fichier`	supprime, sans passer par la corbeille	`Suppr`
`start fichier`	ouvre avec le logiciel associé	double-clic

Le chemin d'un fichier en ligne de commande

En ligne de commande, un fichier se désigne par son chemin, écrit après le nom de la commande. Un chemin **absolu** part de la racine, un chemin **relatif** du dossier où l'on se trouve.

	Windows	macOS et Linux
Le séparateur de dossiers	la barre inversée `\`, *backslash*	la barre `/`, *slash*

```
C:\Users\alice\
└─ cours1\
  ├─ 1a_formats\
  │ └─ depart\
  │   └─ raven.odt
  └─ 1c_programme\
    └─ altitudes.py
```

	Le chemin de `raven.odt`	Depuis
Absolu	`C:\Users\alice\cours1\1a_formats\depart\raven.odt`	la racine
Relatif	`1a_formats\depart\raven.odt`	`cours1`
Relatif qui remonte	`..\1a_formats\depart\raven.odt`	`1c_programme`

Sous macOS et Linux, l'autre séparateur : `/home/alice/cours1/1a_formats/depart/raven.odt`. Un projet qui n'écrit que des chemins relatifs se copie, se déplace et s'envoie sans rien changer : `C:\Users\alice` n'existe que sur un poste.

Les chemins dans Git Bash

Git Bash écrit les chemins à la façon de Linux : des barres obliques, et le disque `C:` devient `/c`.

Dans l'explorateur	Dans Git Bash	
`C:\Users\eleve\Desktop\info01`	`/c/Users/eleve/Desktop/info01`	chemin absolu
`C:\Users\eleve`	`~`	le dossier personnel
le dossier ouvert	`.`	le dossier courant
le dossier au-dessus	`..`	le dossier parent
`..\1a_formats\depart`	`../1a_formats/depart`	chemin relatif

Un chemin copié depuis la barre d'adresse de l'explorateur garde ses `\` : dans Git Bash, il s'écrit entre guillemets.

Fichiers cachés et motif *

Un nom qui commence par un point désigne un fichier caché : `ls` ne l'affiche pas, `ls -a` l'affiche.

```
$ ls travail
$ ls -a travail
./  ../
```

`.` et `..`, le dossier courant et son parent, sont les deux entrées cachées de tout dossier.

Dans un nom de fichier, `*` remplace n'importe quelle suite de caractères.

Commande	Noms désignés dans `depart/`
`ls depart/*.txt`	les deux fichiers `.txt`
`ls depart/raven*`	les trois fichiers de **The Raven**
`cp depart/*.odt travail/`	les deux documents LibreOffice

Git Bash remplace le motif par la liste des noms qui correspondent, puis lance la commande. Le TD 1c crée un fichier caché, `.bash_profile`, dans le dossier personnel ; le cours 2 un dossier caché, `.git`.

TD 1b

Les fichiers du TD 1a en ligne de commande

Objectif : manipuler des fichiers en ligne de commande, dans Git Bash : se déplacer, copier, renommer, ouvrir

`cours1/1b_terminal/` ≈ 15′

Ouvrir Git Bash dans le dossier du TD

1. Dans l'explorateur, ouvrir le dossier `cours1\1b_terminal`. Il contient les dossiers `depart` et `travail`.
2. Faire un clic droit sur un endroit vide de la fenêtre. Choisir « Afficher d'autres options », puis « Open Git Bash here ».
3. Dans la fenêtre de Git Bash, vérifier que la première ligne de l'invite se termine par le dossier du TD, comme sur le schéma.



Si le menu ne propose pas Git Bash : ouvrir Git Bash depuis le menu Démarrer, puis taper `cd ~/Desktop/info01/cours1/1b_terminal`.

Se déplacer et lister

Taper chaque commande dans Git Bash, puis appuyer sur Entrée.

1. Afficher le dossier courant : `pwd`.
2. Lister le contenu du dossier courant : `ls`.
3. Lister le contenu de `depart` : `ls depart`.
4. Lister le contenu de `travail`, entrées cachées comprises : `ls -a travail`.
5. Entrer dans `depart` : `cd depart`. Taper ensuite `pwd`, et relire l'invite.
6. Revenir dans `1b_terminal` : `cd ..`.
7. Sans quitter `1b_terminal`, lister les fichiers exportés au TD 1a : `ls ../1a_formats/travail`.

Pour retrouver une commande déjà tapée, appuyer sur la flèche vers le haut. Pour compléter un nom commencé, appuyer sur `Tab`.

Copier, renommer, ouvrir

`cp` copie, `mv` renomme, `start` ouvre un fichier avec le logiciel que Windows associe à son extension, comme un double-clic.

1. Copier `raven.odt` dans `travail` : `cp depart/raven.odt travail/`.
2. Renommer la copie en `raven_odt.pdf` : `mv travail/raven.odt travail/raven_odt.pdf`.
3. Ouvrir le fichier renommé : `start travail/raven_odt.pdf`. Noter le logiciel qui s'ouvre et ce qu'il affiche.
4. Copier `raven.odt` sous le nom `riri.fifi.loulou.odt`, puis ouvrir la copie :
   ```
   cp depart/raven.odt travail/riri.fifi.loulou.odt
   start travail/riri.fifi.loulou.odt
   ```
5. Copier `raven.odt` sous le nom `raven.loulou`, puis ouvrir la copie. Si Windows demande un logiciel, choisir LibreOffice Writer.
   ```
   cp depart/raven.odt travail/raven.loulou
   start travail/raven.loulou
   ```

Un espace dans un nom de fichier

L'interpréteur coupe la ligne aux espaces. Un nom qui en contient s'écrit entre guillemets.

1. Copier `raven_une_ligne.txt` sous le nom `le corbeau.txt`, sans guillemets, puis lire le message :
   `cp depart/raven_une_ligne.txt travail/le corbeau.txt`
2. Refaire la copie, avec le nom entre guillemets :
   `cp depart/raven_une_ligne.txt "travail/le corbeau.txt"`
3. Lister le contenu de `travail` : `ls travail`.
4. Ouvrir la copie : `start "" "travail/le corbeau.txt"`.

Dans les noms de fichiers du module, _ remplace l'espace.

Copier plusieurs fichiers d'un coup

1. Créer un dossier `textes` dans `travail` : `mkdir travail/textes`.
2. Copier dans ce dossier tous les fichiers `.txt` de `depart` : `cp depart/*.txt travail/textes/`.
3. Lister le contenu de `textes` : `ls travail/textes`.
4. Lister les fichiers `.odt` de `depart` : `ls depart/*.odt`.
5. Supprimer l'une des copies, puis lister de nouveau `textes` :

```
rm travail/textes/auld_lang_syne_une_ligne.txt
ls travail/textes
```

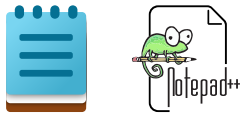
Programme Python et notebook

Un programme Python est un fichier texte, que l'interpréteur python exécute. Un notebook réunit ce code, son résultat et du texte écrit en Markdown.

Rappel des parties 1 et 2

`python` est un programme en ligne de commande. Il prend en entrée un fichier texte, écrit dans un éditeur de texte comme le Bloc-notes, dont l'extension est souvent `.py`.





écrit dans le Bloc-notes ou Notepad++

lancé dans Git Bash :

```
python altitudes.py
```

Partie 1 : un logiciel lit des fichiers en entrée, et l'extension indique leur type. Partie 2 : une commande se tape dans un terminal.

Compiler un programme

Le processeur n’exécute que des instructions machine. Un compilateur traduit une fois le texte du programme en instructions machine, et les enregistre dans un fichier exécutable.

Compilé : C, C++

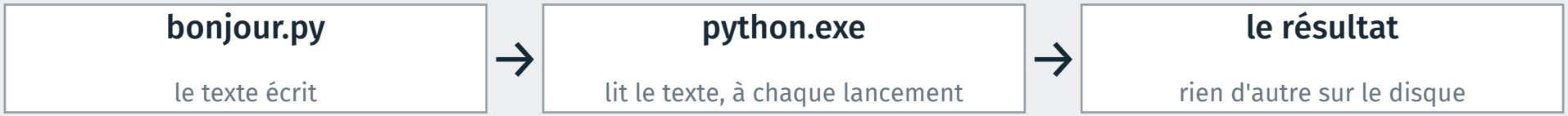


Le fichier exécutable se lance seul, sans le compilateur, sur le système pour lequel il a été compilé.

Interpréter un programme

Un interpréteur lit le texte du programme et l'exécute, à chaque lancement, sans fichier exécutable. Python est un langage interprété.

Interprété : Python



L'interpréteur est lui-même un programme compilé : `python.exe` est écrit en C. Pour lancer `bonjour.py` sur une autre machine, Python doit y être installé.

Python, un programme en ligne de commande

Dans un terminal, la commande `python` suivie du nom d'un fichier exécute ce fichier. Tapée seule, elle ouvre une session interactive, où Python exécute chaque ligne dès qu'on appuie sur Entrée.

Exécuter un fichier : `python altitudes.py`

○○○ Git Bash

```
$ cd ~/Desktop/info01/cours1/1c_programme
$ python altitudes.py
moyenne : 129.0 m
```

Ouvrir une session interactive : `python`

○○○ Git Bash

```
$ python
Python 3.12.14 (main, Sep  2 2026, 23:27:36) [GCC
15.3.0] on linux
>>> 128.4 + 131.0
259.4
>>> exit()
```

Python écrit ses résultats dans le terminal, sans ouvrir de fenêtre. Comme `ls` ou `cp`, `python` est une commande, qu'un script peut lancer.

Exécution d'un programme Python

La commande `python altitudes.py` lance l'interpréteur, qui lit le fichier enregistré et l'exécute. `which python` affiche l'emplacement de cet interpréteur. Pour en lancer un autre, on tape le chemin complet de son fichier `python.exe`.



Dans Git Bash : le Python par défaut, puis celui d'Anaconda

Git Bash

```
$ which python
/c/Python27/python
$ /c/ProgramData/anaconda3/python.exe altitudes.py
moyenne : 129.0 m
```

Exemple d'un poste où un Python 2.7 est installé dans `C:\Python27`. Dans Git Bash, `C:\` s'écrit `/c/`, et chaque `\` devient `/`.

Programmation littérale

Un notebook réunit le texte, le code et son résultat dans un seul document.



Le terme « programmation littérale » est de Donald Knuth, 1984.

TD 1c

Écrire et lancer un programme

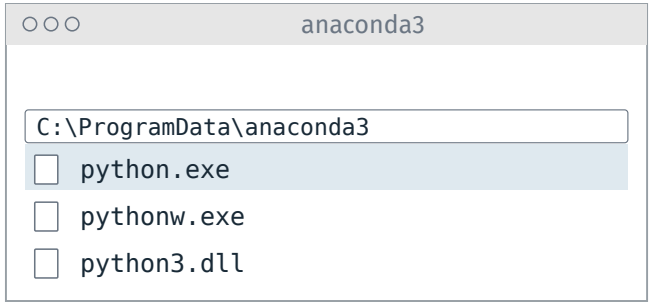
Objectif : lancer un programme Python depuis le terminal, en sachant quel Python l'exécute, puis utiliser Python en interactif

`cours1/1c_programme/` ≈ 20′

Quel programme python la commande lance

1. Aller dans le dossier du TD : `cd ../1c_programme`. Git Bash est encore ouvert dans `1b_terminal`.
2. Afficher le fichier que lance la commande `python` : `which python`.
3. Ouvrir le dossier d'Anaconda dans l'explorateur : taper son chemin dans la barre d'adresse, comme sur le schéma, puis Entrée.
4. Lancer ce Python par son chemin complet :
   `/c/ProgramData/anaconda3/python.exe --version`

Dans Git Bash, `C:\` s'écrit `/c/`, et chaque `\` devient `/`.



Python dans Git Bash, une fois par poste

`conda init bash` règle Git Bash pour qu'il active conda à chaque ouverture.

1. Configurer conda pour Git Bash :
   ```
   source /c/ProgramData/anaconda3/etc/profile.d/conda.sh
   conda init bash
   ```
2. Fermer Git Bash, puis le rouvrir dans `1c_programme` : clic droit dans le dossier, Open Git Bash here.
3. Vérifier le Python lancé : `which python`, puis `python --version`.
4. Afficher les fichiers cachés du dossier personnel : `ls -a ~`.



Si le réglage échoue : invite de commandes d'Anaconda, dans le menu Démarrer.

Le programme du TD

Le programme `altitudes.py` calcule la moyenne de trois altitudes, puis l'affiche.

`altitudes.py`

```
altitudes = [128.4, 131.0, 127.6]
total = 0
for altitude in altitudes:
    total = total + altitude
moyenne = total / len(altitudes)
print(f"moyenne : {moyenne:.1f} m")
```

Lancé en entier

```
$ python altitudes.py
moyenne : 129.0 m
```

Une seule ligne de sortie, celle du `print` final. Les étapes du calcul ne s'affichent pas.

Sortie réelle, Python 3.12. La moyenne de 128,4, 131,0 et 127,6 vaut bien 129,0.

Lancer, modifier, relancer

1. Ouvrir `altitudes.py` dans Notepad++ : Fichier, Ouvrir, puis `cours1\1c_programme\altitudes.py`.
2. Lancer le programme dans Git Bash : `python altitudes.py`.
3. Dans Notepad++, ajouter `133.2` à la liste, sans enregistrer :
   `altitudes = [128.4, 131.0, 127.6, 133.2]`
4. Relancer le programme dans Git Bash. Flèche vers le haut, puis Entrée.
5. Enregistrer dans Notepad++ (`Ctrl` + `S`), puis relancer le programme.

Python en interactif

1. Ouvrir une session interactive dans Git Bash : `python`.
2. Taper une à une les trois premières lignes de `altitudes.py`, comme dans la session à droite. Après la ligne indentée, appuyer une fois de plus sur Entrée.
3. Afficher la valeur de `total` : taper `total`.
4. Calculer la moyenne : `total / len(altitudes)`.
5. Quitter la session : `exit()`.

Git Bash

```
$ python
Python 3.12.14 (main, Sep  2 2026, 23:27:36) [GCC 15.3.0] on linux
>>> altitudes = [128.4, 131.0, 127.6]
>>> total = 0
>>> for altitude in altitudes:
...     total = total + altitude
...
>>> total
387.0
>>> total / len(altitudes)
129.0
>>> exit()
```

Session réelle. Dans une session interactive, une variable tapée seule affiche sa valeur, sans `print`.

Lancer le script, si le temps le permet

Un script est un fichier texte qui contient des commandes. `bash` les exécute dans l'ordre, comme si on les tapait.

1. Ouvrir `commandes.sh` dans Notepad++, et lire ses six lignes, reproduites ci-dessous.
2. Lancer le script dans Git Bash : `bash commandes.sh`.
3. Relancer le script : flèche vers le haut, puis Entrée.

`commandes.sh`

```
# Commandes du TD 1b, puis le programme du TD 1c.
# Lancer depuis 1c_programme : bash commandes.sh
mkdir -p copies
cp ../1b_terminal/depart/*.txt copies/
ls copies
python altitudes.py
```

Le même script pour l'invite de commandes, `commandes.bat` : annexe du guide.

TD 1d

Un notebook dans JupyterLab

Objectif : exécuter un notebook dans JupyterLab, voir ce que le noyau retient d'une cellule à l'autre, et documenter le notebook en Markdown

`cours1/1d_notebook/` ≈ 12′

Lancer JupyterLab

JupyterLab est installé avec Anaconda, et se lance dans un terminal où conda est actif. Le module donne deux terminaux, pour ne pas être bloqué si le réglage de Git Bash manque sur un poste.

Depuis l'invite de commandes d'Anaconda

```
(base) …>cd Desktop\info01\cours1\1d_notebook
(base) …>jupyter lab
```

Menu Démarrer, « Anaconda Prompt ». conda y est actif sans réglage : à essayer d'abord.

Depuis Git Bash

```
$ cd ~/Desktop/info01/cours1/1d_notebook
$ jupyter lab
```

Le terminal du module, après le réglage du TD 1c.

Le navigateur s'ouvre sur `localhost:8888`, le poste lui-même : le serveur de JupyterLab tourne sur le poste, et le terminal reste occupé tant qu'il tourne. `Ctrl` + `C` l'arrête. Le cours 5 y revient.

Exécuter les cellules

`altitudes.ipynb` contient le programme du TD 1c, découpé en cellules. Pour exécuter une cellule, cliquer dedans, puis taper `Maj` + `Entrée`.

	Ce qu'il faut faire
1	invite de commandes d'Anaconda, `cd Desktop\info01\cours1\1d_notebook`, `jupyter lab` ; ouvrir `altitudes.ipynb`
2	exécuter les cellules une à une, de haut en bas
3	exécuter une seconde fois la cellule de la boucle, puis celle de la moyenne
4	Kernel → Restart Kernel and Run All Cells

Le numéro entre crochets donne l'ordre dans lequel les cellules ont été exécutées.

La syntaxe minimale de Markdown

Une cellule de texte s'écrit en Markdown : du texte brut, où quelques signes indiquent la mise en forme.

Ce qu'on tape	Ce qui s'affiche
`# Altitudes`, `## Les données`	**Altitudes** **Les données**
deux lignes séparées par une ligne vide	deux paragraphes
`du *texte* en **gras**`	du *texte* en **gras**
`- une puce`	• une puce
`1. une étape`	1. une étape
`[le site](https://jupyter.org)`	le site
la variable `total`	la variable `total`

Le texte tapé reste lisible sans être affiché mis en forme. Le cours 2 y ajoute les tableaux et les images.

Ajouter une cellule de texte

Une cellule de texte en tête du notebook, écrite en Markdown, décrit ce que fait le programme.

	Ce qu'il faut faire
1	cliquer sur la première cellule, touche `Échap`, puis `A`
2	touche `M`, ou « Markdown » dans la liste en haut de la page
3	y écrire un titre, une phrase, la liste des données, un lien
4	`Maj` + `Entrée` ; double-clic pour revenir au texte
5	`Ctrl` + `S`
6	si le temps le permet : `Ctrl` + `C` dans l'invite, puis `jupyter lab` depuis Git Bash

À retenir

Un logiciel	transforme une entrée en sortie ; un fichier conserve le résultat
Une extension	nomme le type du fichier, elle ne modifie pas son contenu
Le disque du poste	garde vos fichiers ; le dossier partagé est à toute la salle
Le terminal	transmet des commandes à un interpréteur ; au module, Git Bash
Un chemin relatif	part du dossier courant ; `..` remonte d'un dossier
Une commande	désigne un fichier exécutable, cherché dans PATH
L'interpréteur python	exécute le fichier enregistré sur le disque
Automatiser	un logiciel pour ce qu'il prévoit, un script de commandes pour répéter, un programme pour le reste
Un notebook	texte, code et résultats ; le noyau retient les variables